Class 16: Feature Engineering — From Raw Data to Validated Features

Data-driven Systems Engineering

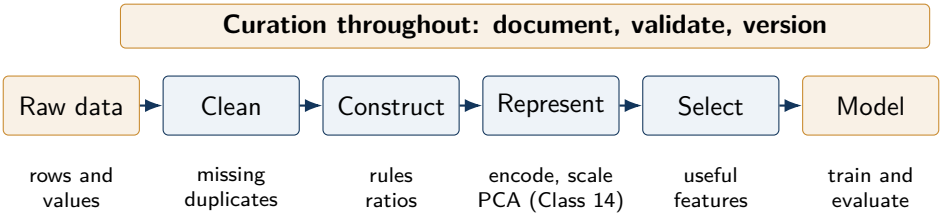
Agenda

- Data curation and cleaning: missing values, duplicates, invalid values, outliers.
- Feature construction and representation: domain knowledge, encoding, scaling, PCA.
- Feature selection: filter and wrapper methods.
- Leakage checks and the Titanic case: baseline, ablation, reproducible pipeline.

Encoding as part of feature engineering

- Encoding is one component of feature engineering, alongside cleaning, construction, scaling, and selection.
- Data cleaning addresses missing values, duplicates, and inconsistent types; encoding determines how categories are represented numerically.
- Not every available variable should become a feature: some are redundant, some carry only noise, and some induce leakage.
- Informative features are frequently absent from the raw data and must be constructed from domain knowledge.

Feature engineering in the modeling workflow



Feature construction and transformation

Construction creates candidate features (e.g., ratios or rolling means). It is one kind of feature transformation; encoding and scaling also transform features, while selection chooses which candidates to keep.

The workflow is iterative: reject leaking or unavailable features early; construction and selection may be revisited. PCA constructs components rather than selecting original columns.

Raw data: the starting point



What is raw data?

Data exactly as collected or exported from its source, before cleaning, construction, or encoding. Each row is an observation; each column is an attribute.

- **Types:** numeric, categorical, text, timestamp, stream.
- **Granularity:** what one row represents.
- **Provenance:** source, date, version.

What to expect in raw data

- Missing and duplicated values
- Inconsistent types or units
- Outliers and invalid entries
- Identifiers and leaking columns

Good practice

Keep an untouched copy; apply every change in code.

Running case: Titanic survival prediction

Engineering question

Can we predict whether a passenger survives, using only information available at the intended prediction moment?

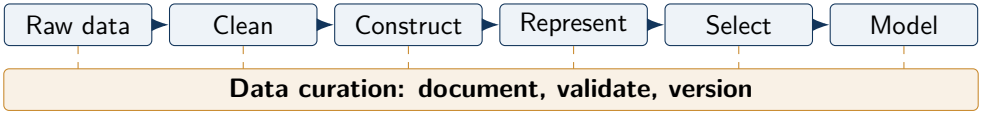
- **Target:** `survived` (binary), $n = 891$ passengers.
- **Source:** classic Kaggle training set: 12 columns, including the target and passenger identifier.
- **Candidates:** 10 original passenger attributes; derived features must have explicit definitions.

Evidence we will reveal

Raw $\rightarrow$ Curate $\rightarrow$ Clean $\rightarrow$ Construct $\rightarrow$ Represent $\rightarrow$ Select $\rightarrow$ Validate.

Class discussion: Is a variable useful just because it is predictive?

Data curation



- Curation makes a dataset understandable and reusable beyond its original authors.
- A data dictionary records the meaning, unit, and valid range of every column.
- Capture provenance and lineage: source, extraction/query, timestamp, version, and feature definitions; an extraction date alone does not guarantee reproducibility.
- Validation rules (schema and range checks) detect upstream problems before they reach the modeling stage.

Repository example

For Titanic, record a feature contract: definition, type, allowed values, missingness policy, source, prediction-time availability, and version. The fraud-case specification in `documents/FraudDetectionAsaService.pdf` illustrates similar documentation.

Titanic / 1: curate the feature contract

Field	Type	Meaning / validation	Prediction-time audit
`survived`	binary	Target; 0 or 1	Never an input
`sex`	nominal	Passenger category	Check available
`age`	numeric	Age in years; missing possible	Check available
`sibsp`	count	Siblings/spouses aboard; ≥ 0	Check available
`parch`	count	Parents/children aboard; ≥ 0	Check available
`embarked`	nominal	Embarkation port; some missing	Check available
`passengerid`	identifier	Unique passenger key	Exclude from baseline inputs

What should be versioned?

Dataset snapshot / extraction, column schema, validation rules, cleaning and construction code, split definition, and fitted preprocessing artifacts.

Question: Which fields would you reject *before* looking at feature scores?

Data cleaning: scope and quality dimensions



Missingness
Measure rates and patterns; check whether absence is meaningful. Choose to preserve, impute, or remove with justification.

Duplicates
Check exact and near repeats and key uniqueness. Confirm that repeated rows represent the same entity before deduplicating.

Invalid or inconsistent values
Check schema, types, labels, units, formats, and allowed ranges. Standardize valid variants; correct or reject invalid values.

Outliers
Flag unusual values with domain limits and robust summaries. Investigate each: an extreme may be valid, erroneous, or a measurement fault.

Cleaning is a documented decision, not automatic deletion. For each issue, record the evidence found, the action taken (keep, correct, impute, or remove), and why that action is justified.

Data cleaning: common techniques

Issue	Detect	Treat
Missing values	Missingness rates and matrix plots; Little's MCAR test	Mean/median/mode imputation; k-NN imputation; iterative imputation (MICE); missing-value indicator
Duplicates	Key-uniqueness checks; hashing for exact matches; string similarity (Levenshtein, Jaro–Winkler) for near matches	Record linkage with blocking; MinHash/LSH at scale; keep-first or merge rule
Invalid or inconsistent	Schema and type validation (e.g., pandera, Great Expectations); regular expressions; range and unit checks	Type coercion; category mapping to canonical labels; unit conversion; reject or flag
Outliers	Tukey IQR fences; z-score and robust z-score (MAD); Isolation Forest; Local Outlier Factor	Winsorizing (capping); log or Box–Cox transform; robust scalers; remove only if erroneous

Record and validate

Document the checks and decisions. Fit imputation within each training fold and apply it unchanged to held-out and production data.

Titanic / 2: cleaning decisions per column

Original column	Candidate policy	Assessment / rationale
`cabin`	Preserve missingness	Often absent; derive a known/unknown indicator or a deck category with an explicit Unknown level.
`age`	Median imputation	Inspect missingness and distributions; fit the median within each training fold.
`embarked`	Mode or Unknown	Inspect missing entries; fit any mode using the training fold.
`fare`	Validate / impute	Check range, extremes, and missingness in each supplied file.
`passengerid`	Validate uniqueness	Audit repeated identifiers; equal passenger attributes alone do not establish duplicate records.

Feature construction from domain knowledge



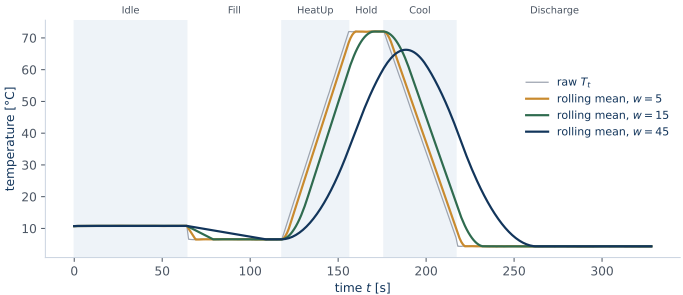
What is feature construction?

Deriving new variables from existing ones, guided by knowledge of the problem, so that a useful pattern becomes easier for the model to use.

Main methods

- Arithmetic combinations
- Aggregations
- Binning
- Indicators and rules
- Date and time parts
- Text extraction
- Interactions
- Rolling windows

Streams and sensors: rolling mean and window size



One batch generated by `pasteurization/synth_sensors.py`.

Example with $w = 3$:

t	1	2	3	4	5
x_t	10	12	14	20	18
$\bar{x}_t$	–	–	12	15.3	17.3

$\bar{x}_4 = (12 + 14 + 20)/3$; the first $w - 1$ values are undefined.

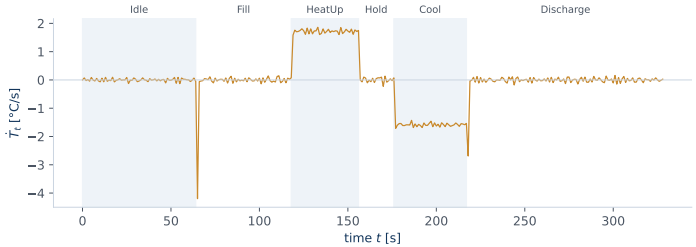
Rolling mean of the temperature T_t:

$$\bar{T}_t = \frac{1}{w} \sum_{i=0}^{w-1} T_{t-i}$$

- Small w (5): follows the signal closely, little smoothing.
- Medium w (15): removes noise, with a visible delay.
- Large w (45): very smooth, but lags by about $w/2$ samples and flattens the Hold plateau.

Choose w to trade noise reduction against delay.

Streams and sensors: rate of change



One batch generated by `pasteurization/synth_sensors.py`.

Example with $\Delta t = 1$ s:

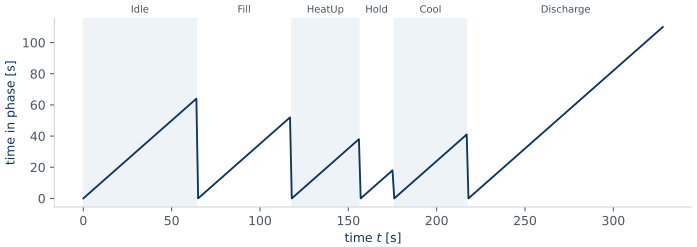
t	1	2	3	4	5	6
state	Idle	Idle	HeatUp	HeatUp	HeatUp	Hold
T_t	20	20	22	25	28	28
$\dot{T}_t$	–	0	2	3	3	0

Rate of change:

$$\dot{T}_t = \frac{T_t - T_{t-1}}{\Delta t}$$

- Positive in HeatUp, negative in Cool, near zero elsewhere.
- The raw value T_t cannot separate these regimes: 28 °C may be heating or cooling.
- The first sample is undefined: it has no predecessor.

Streams and sensors: time in phase



One batch generated by `pasteurization/synth_sensors.py`.

Example with $\Delta t = 1$ s:

t	1	2	3	4	5	6
state	Idle	Idle	HeatUp	HeatUp	HeatUp	Hold
time in phase [s]	0	1	0	1	2	0

Time in phase: seconds since the last change of state.

- The counter resets to 0 whenever the state changes.
- It distinguishes a slow heating from a normal one: the same temperature after 10 s or 40 s in HeatUp means different things.
- It uses only past observations, so it can be computed online, one sample at a time.
- The state label is categorical: it can be encoded as in Class 14 (e.g., one-hot) and used beside this feature.

Represent candidates before selection



- Selection methods need features in a representation they can use: encode categorical variables and scale numeric ones when the method is scale-sensitive.
- One-hot, ordinal, target, and cyclic encodings offer different representations; the choice depends on the variable and the model.
- PCA transforms the feature space into components. It constructs a new representation rather than selecting original columns.
- Fit imputers, scalers, PCA, target-dependent selectors, and other learned transformations using training folds only; use out-of-fold target encoding.

Feature selection: filter and wrapper

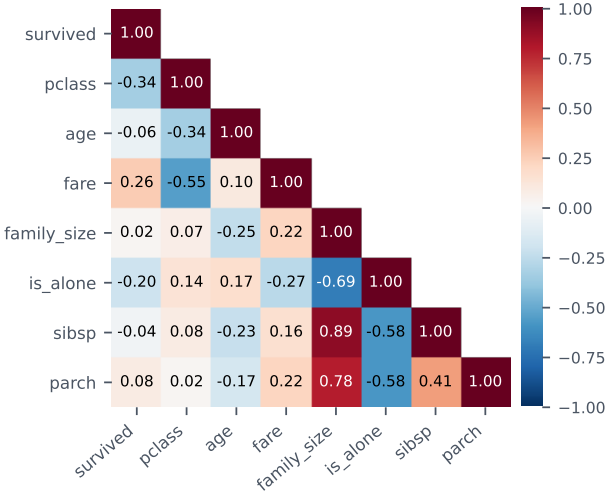


Given d candidate features, selection seeks a subset $S \subseteq \{1, \ldots, d\}$ that maximizes a criterion $J(S)$. The two families differ in how J is defined.

Filter J is a model-independent statistic computed for each feature, e.g. correlation, mutual information, or the χ^2 statistic.

Wrapper $J(S)$ is the validation performance of a model trained on S; subsets are explored by a search strategy (e.g. forward or backward selection).

Titanic / 6: engineering changes feature dependencies



- Pearson correlations after cleaning (age imputed with the median).
- `family_size` is strongly correlated with its components `sibsp` (0.89) and `parch` (0.78): keeping all three adds collinearity.
- `is_alone` has a larger linear association with `survived` (-0.20) than `family_size` (0.02).
- Pearson measures only linear dependence; a near-zero value does not make a feature uninformative.
- **Spearman** is Pearson on ranks: it captures monotonic association and resists outliers. Prefer it for skewed, ordinal, or curved data; use Pearson for roughly linear data.

Modeling: what it is and main types



What is a model?

A function f_θ that maps features to a prediction, $\hat{y} = f_\theta(\mathbf{x})$, with parameters θ estimated from data. Training and evaluation are the subject of Class 18.

By learning paradigm

- **Supervised:** learns from labeled pairs $(\mathbf{x}_i, y_i)$; classification or regression. Titanic: predict `survived`.
- **Unsupervised:** no target; finds structure, e.g. clustering or PCA (Class 14).
- **Self-supervised:** the target is built from the data itself, e.g. word2vec (Class 14).
- **Reinforcement:** learns from rewards obtained by interacting with an environment.

By model family

- **Linear:** linear and logistic regression; interpretable.
- **Tree-based:** decision trees, random forests, gradient boosting; capture non-linearity.
- **Instance-based:** k-nearest neighbours.
- **Neural networks:** multilayer perceptrons, CNNs, RNNs; see Class 13 for families by data type.

Ablation: measuring what each component contributes

What is an ablation?

A controlled experiment that removes (or adds) one component of a system, such as a feature, a feature group, or a preprocessing step, and measures how the performance changes.

With a score S (higher is better), the contribution of a component c is

$$\Delta_c = S(\text{full}) - S(\text{full without } c).$$

Two directions

- **Backward (remove):** start from the full set and drop one component at a time.
- **Forward (add):** start from a baseline and add one component at a time.

Principles

- Change one thing at a time; keep data, model, and metric fixed.
- Repeat the measurement and report the variation.
- Δ_c holds *in this context*: correlated features compensate for each other.

Good practices in feature engineering

Before modeling

- **Raw data:** keep an untouched copy; apply every change in code.
- **Curation:** maintain a data dictionary, provenance, and validation rules.
- **Cleaning:** record evidence, action, and rationale for each issue.
- **Construction:** state each feature's rationale in one sentence and check redundancy with its source variables.
- **Selection:** fix the criterion before looking at results; prefer Spearman for monotonic, non-linear relations.

Avoiding leakage

- Is the feature available at prediction time, or does it use information from the future?
- Does it encode a post-decision outcome, such as the result of a manual review?
- Is it computed identically in production and in the notebook?
- Does it depend on joins or upstream systems that may be unavailable at inference time?
- For streams, use only past observations (causal windows).

The complete taxonomy of leakage (target, preprocessing, temporal, group, duplicate, model selection) was presented in Class 13. Measure each feature's contribution with an ablation.

Repository connections

Repo activity

- `notebooks/Class4_DataEngineering.ipynb` illustrates cleaning and selection using the Seaborn variant. Its extra columns and reported model scores should not be transferred to the Kaggle case without rerunning the analysis.
- `pasteurization/synth_sensors.py` provides a contrastive online feature example (`dTdt`); rolling means must use past/current observations only.
- `streamlit/modeling.py` saves the fitted scaler together with the model, illustrating that preprocessing decisions must be persisted with the model rather than applied once in a notebook.

Papers and materials

Papers and materials

- Guyon, I., Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Kohavi, R., John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Little, R. J. A., Rubin, D. B. (2019). *Statistical Analysis with Missing Data*, 3rd ed. Wiley.
- Gebru, T., Morgenstern, J., Vecchione, B., Wortman Vaughan, J., Wallach, H., Daumé III, H., Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
- Zheng, A., Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.
- scikit-learn User Guide: feature selection; pipelines and composite estimators.

Take-away message

Features are design decisions:

curate and validate data; construct with domain knowledge;
fit transformations and selection within training folds;
deploy only reproducible, available, monitored features.

Thank you for your attention.

Next: Class 17 — version control with Git.